COMP3028 | SOFTWARE CONSTRUCTION

# WORKSHOP 8
## Node.js Data Dive

From SQL queries to a working Express API




**Audience:** Absolute beginners with basic JavaScript knowledge. **Suggested duration:** 120–150 minutes.
**Final product:** A small, working Node.js server that reads users from PostgreSQL, retrieves one user by ID, searches for users, and creates new users using safe SQL queries.

**How to use this guide.** Complete the steps in order. Type the commands and code yourself, save each file, and compare your screen with the expected results. A result that differs from the example is not always an error: database-generated IDs can change as you add records.

**Learning checkpoint**

By the end of the workshop you should be able to:

1. Explain how an HTTP request reaches a database through an Express route.
2. Create a PostgreSQL table and insert sample data.
3. Use the `postgres` library and `await sql`...`` to query PostgreSQL.
4. Pass JavaScript values safely into queries through bound parameters.
5. Insert a JavaScript object with `sql(user)` and return the inserted row.
6. Test GET and POST routes and diagnose common connection and syntax errors.

## Contents

**1 Before you begin** **3**
1.1 What the original workshop covers . . . 3
1.2 Software you need . . . 3
1.3 Check your installation . . . 3
1.4 Your project at a glance . . . 4

**2 Part A – Build the PostgreSQL database** **4**
2.1 Create a database . . . 4
2.2 Create the users table . . . 4
2.3 Check your understanding . . . 5

**3 Part B – Create your Node.js project** **5**

3.1 Initialise npm and install packages . . . 5
3.2 Configure your database connection . . . 6
3.3 Make a reusable connection file . . . 6

**4 Part C – Understand SQL tagged templates 7**
4.1 Ordinary strings versus tagged templates . . . 7
4.2 Read all rows . . . 7
4.3 Pass values safely with bound parameters . . . 7
4.4 Insert an object with matching field names . . . 8

**5 Part D – Create the Express API 8**
5.1 What is an API route? . . . 8
5.2 Type the complete server . . . 8

**6 Part E – Run and test each route 11**
6.1 Start the server . . . 11
6.2 Test 1: Verify the server is reachable . . . 11
6.3 Test 2: Verify the PostgreSQL connection . . . 11
6.4 Test 3: Read all records . . . 12
6.5 Test 4: Read one record with a bound parameter . . . 12
6.6 Test 5: Search by a name . . . 12
6.7 Test 6: Insert a user through POST . . . 12

**7 Part F – Trace what happens behind the scenes 13**
7.1 Mini experiment: inspect a SQL value . . . 14

**8 Part G – Troubleshooting for beginners 14**

**9 Part H – Independent practice 15**
9.1 Exercise 1: Query a new user (easy) . . . 15
9.2 Exercise 2: Filter users by minimum age (medium) . . . 15
9.3 Exercise 3: Use a bound query value (medium) . . . 15
9.4 Exercise 4: Reflect on security (short answer) . . . 15

**10 Part I – Self-assessment and submission 16**
10.1 Reflection prompts . . . 16

**11 Quick-reference sheet 16**

# 1 Before you begin

## 1.1 What the original workshop covers

The attached *Workshop 8: Node.js Data Dive* slides focus on four ideas: writing SQL in backticks using Postgres.js tagged templates; passing values as bound parameters; inserting records from JavaScript objects; and experimenting with HTTP routes. This guide expands those ideas into a standalone, hands-on exercise. The installation, schema, folder structure, sample routes and testing steps are added for beginners.

**Optional documentation:** https://github.com/porsager/postgres, https://expressjs.com/ and https://www.postgresql.org/docs/current/tutorial.html.

**Important**

**Security clarification.** The slides associate bound SQL parameters with protection from cross-site scripting. Their primary purpose here is to prevent *SQL injection* by treating input as data rather than executable SQL. Cross-site scripting (XSS) is a separate web-output security issue.

## 1.2 Software you need

| Tool | Purpose and action |
|---|---|
| Node.js (LTS) and npm | Run the JavaScript server and install dependencies. Download from https://nodejs.org/. |
| Visual Studio Code | Create and edit project files. https://code.visualstudio.com/ |
| PostgreSQL | Store the `users` table. https://www.postgresql.org/download/ |
| pgAdmin or `psql` | Execute SQL commands. pgAdmin is commonly bundled with PostgreSQL installers. |
| REST Client extension (recommended) | Send GET and POST requests directly from `requests.http` in VS Code. An alternative is Postman. |

## 1.3 Check your installation

**Step 1.** Open a terminal (VS Code: **Terminal** → **New Terminal**). Type these commands, pressing Enter after each one.

```
node --version
npm --version
```

**Expected result.** Both commands print version numbers. Any supported, modern Node.js LTS release is suitable for the activities.

**Step 2.** Confirm that PostgreSQL is installed and its service is running. Open **pgAdmin**, or enter the following if its command-line client is on your PATH:

```
psql --version
```

**Expected result.** You can open pgAdmin and connect to a PostgreSQL server, or you can see a `psql` version in the terminal.

**Helpful note**

**If the terminal cannot find `psql`:** You can still complete this workshop entirely through pgAdmin's Query Tool. On Windows, a PostgreSQL installation may provide `psql` without adding it to the system PATH.

### 1.4 Your project at a glance

You will create the following files. Keep all files in the `workshop8-data-dive` folder.

```
workshop8-data-dive/
|-- .env              # Your local database configuration
|-- .gitignore        # Keep secrets out of Git
|-- schema.sql        # Table creation and sample data
|-- db.js             # PostgreSQL connection
|-- server.js         # Express application and routes
|-- requests.http     # Repeatable API test requests
|-- package.json      # Created by npm
`-- node_modules/     # Created automatically by npm
```

## 2 Part A – Build the PostgreSQL database

### 2.1 Create a database

**Step 1.** Open pgAdmin and connect to your local PostgreSQL server. Enter your server password if prompted. This password is the one you configured during PostgreSQL installation.

**Step 2.** In the left-hand browser panel, select the existing maintenance database named **postgres**. Open **Tools** → **Query Tool**. Run:

```
CREATE DATABASE workshop8_db;
```

**Expected result.** pgAdmin reports that the query completed successfully. In its browser panel, refresh **Databases**; you should see `workshop8_db`.

**Helpful note**

If you receive *database already exists*, do not run the command again. Use the existing `workshop8_db` database.

**Step 3.** In the left-hand panel, select **workshop8__db**, then open a *new* Query Tool for that database. This matters: the following table must be created inside `workshop8_db`, not inside `postgres`.

### 2.2 Create the users table

**Step 4.** In VS Code, create a folder named `workshop8-data-dive`. Inside it, create `schema.sql` and type:

Listing 1: schema.sql – create a table and initial records

```
CREATE TABLE IF NOT EXISTS users (
  user_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  age INTEGER NOT NULL CHECK (age BETWEEN 0 AND 130)
```

```
);

INSERT INTO users (name, age)
VALUES ('Jane', 27), ('Joe', 26);

SELECT user_id, name, age
FROM users
ORDER BY user_id;
```

**Step 5.** Copy the content of `schema.sql` into the **workshop8_db** Query Tool and click the **Execute** button (or press the tool's run shortcut).

**Expected result.** With a fresh table, the final query returns a grid similar to:

| user_id | name | age |
|---|---|---|
| 1 | Jane | 27 |
| 2 | Joe | 26 |

**Why this works.** The identity column automatically generates unique user IDs. `NOT NULL` makes names and ages required; `CHECK` restricts ages to 0–130.

**Important**

The `INSERT` command adds two *new* records each time you run it. Do not rerun the entire script casually if you want exactly two users. For a fresh practice database, you may reset its sample data with `TRUNCATE TABLE users RESTART IDENTITY;` before running the insert again. This deletes existing rows, so use it only on your disposable workshop database.

### 2.3 Check your understanding

a) What is a row? What is a column in the `users` table?

b) Why does the table use `user_id` rather than a person's name as its primary key?

c) Try `SELECT name FROM users;` in pgAdmin. How is the result different?

## 3 Part B – Create your Node.js project

### 3.1 Initialise npm and install packages

**Step 1.** In VS Code, select **File** → **Open Folder** and open `workshop8-data-dive`.

**Step 2.** Open a new integrated terminal. Check you are inside that folder. Then type:

```
npm init -y
npm install express postgres dotenv
npm pkg set type=module
npm pkg set scripts.start="node server.js"
```

**Expected result.** npm creates `package.json`, `package-lock.json` and `node\_modules`. The project's `package.json` now has `"type": "module"` and a `start` script.

**What the packages do:**

- `express`: receives HTTP requests and returns responses.

- `postgres`: the Postgres.js library; sends SQL queries to PostgreSQL.
- `dotenv`: loads local configuration from `.env`.

## 3.2 Configure your database connection

**Step 3.** Create a file named exactly `.env` (including the leading dot). Replace the password placeholder in the example with your actual PostgreSQL password:

Listing 2: .env – replace the placeholder password

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=workshop8_db
PGUSER=postgres
PGPASSWORD=YOUR_POSTGRES_PASSWORD
PORT=3000
```

**Helpful note**

The default PostgreSQL port is often 5432. If your setup uses another port, update `PGPORT`. If you created a different PostgreSQL login, use its username and password instead of `postgres`.

**Step 4.** Create `.gitignore`:

Listing 3: .gitignore – files you should not commit

```
node_modules/
.env
```

**Important**

Never publish `.env` or paste a real database password into source code, screenshots or assignments. If you accidentally expose a password, change it.

## 3.3 Make a reusable connection file

**Step 5.** Create `db.js`. Type the complete code below, including both imports and the export:

Listing 4: db.js – connect Node.js to PostgreSQL

```
import 'dotenv/config';
import postgres from 'postgres';

const sql = postgres({
  host: process.env.PGHOST,
  port: Number(process.env.PGPORT),
  database: process.env.PGDATABASE,
  username: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  max: 5
});

export default sql;
```

**Why this works.** `postgres({...})` creates a client used to run SQL. `export default sql` makes it accessible to `server.js`. The library opens connections as needed when a query is executed; constructing the client alone does not verify connectivity.

**Learning checkpoint**

**Checkpoint B** Confirm that `.env`, `.gitignore`, `db.js`, and `package.json` are saved in the same project folder. You have not started the server yet.

# 4 Part C – Understand SQL tagged templates

## 4.1 Ordinary strings versus tagged templates

A normal JavaScript string is just text. Postgres.js lets you write SQL inside *backticks* that are directly preceded by the `sql` tag. For example:

```
const rows = await sql`SELECT * FROM users`;
```

Here, `sql` is the library's query function. The backticks are the JavaScript template-literal syntax. `await` waits until PostgreSQL returns its result.

## 4.2 Read all rows

```
const rows = await sql`
  SELECT user_id, name, age
  FROM users
  ORDER BY user_id
`;
```

**Expected result.** In a fresh sample database, `rows` contains objects equivalent to:

```
[
  { user_id: 1, name: 'Jane', age: 27 },
  { user_id: 2, name: 'Joe', age: 26 }
]
```

Postgres.js returns an array-like result. Each returned database row has properties matching the selected SQL column names.

## 4.3 Pass values safely with bound parameters

Suppose a request supplies `userId = 1`. Use `${userId}` inside the tagged template:

```
const userId = 1;
const rows = await sql`
  SELECT user_id, name, age
  FROM users
  WHERE user_id = ${userId}
`;
```

**Meaning:** PostgreSQL compares `user_id` to the data value supplied by JavaScript. Postgres.js binds the value separately from the SQL command structure.

**Important**

**Avoid unsafe string concatenation:**

```
// DO NOT use user input to build SQL this way:
const query = 'SELECT * FROM users WHERE user_id = ' + input;
```

In this workshop, use `sql‘...${value}...‘` for values. Do not confuse this with plain string interpolation or SQL text concatenation.

### 4.4 Insert an object with matching field names

Postgres.js can generate an insert's column/value section from a JavaScript object:

```
const user = { name: 'John', age: 30 };
const rows = await sql`
  INSERT INTO users ${sql(user)}
  RETURNING user_id, name, age
`;
```

The object's `name` and `age` keys must match database columns. `RETURNING` asks PostgreSQL to send back the inserted record, including its generated ID. You will use this technique in a POST route.

## 5 Part D – Create the Express API

### 5.1 What is an API route?

A route connects an HTTP method and a URL path to a JavaScript function. For example, `GET /users` means "when the client requests /users, run this function and send a response."

| Method and route | Operation | Expected response |
|---|---|---|
| `GET /` | Welcome | A short JSON message. |
| `GET /health` | Connection check | A confirmation that a SQL query succeeds. |
| `GET /users` | Read all | An array of user records. |
| `GET /users/search?name=Ja` | Search names | Users with names containing `Ja`. |
| `GET /users/1` | Read one | One user or a not-found error. |
| `POST /users` | Create | The newly inserted user (HTTP 201). |

### 5.2 Type the complete server

**Step 1.** In the project folder, create `server.js`. Type all of the following code in sequence. Do not skip any section.

Listing 5: server.js – complete application, part 1: initialise and read

```
import express from 'express';
import sql from './db.js';

const app = express();
const port = Number(process.env.PORT || 3000);

// Allows Express to read JSON sent in request bodies.
app.use(express.json());

app.get('/', (req, res) => {
  res.json({ message: 'Welcome to Workshop 8!' });
});

// Check that Node.js can actually query PostgreSQL.
```

```
app.get('/health', async (req, res) => {
  try {
    const rows = await sql`SELECT 1 AS ok`;
    res.json({ message: 'Database connected', result: rows[0] });
  } catch (error) {
    console.error('Health check failed:', error);
    res.status(500).json({ error: 'Database connection failed' });
  }
});

// Return every user, sorted by ID.
app.get('/users', async (req, res) => {
  try {
    const rows = await sql`
      SELECT user_id, name, age
      FROM users
      ORDER BY user_id
    `;
    res.json(rows);
  } catch (error) {
    console.error('GET /users failed:', error);
    res.status(500).json({ error: 'Could not retrieve users' });
  }
});
```

**Step 2.** Add the following code *underneath* the `/users` route, in the same file. Search is written before `/users/:id` so it is matched as a named path rather than treated as an ID.

Listing 6: server.js – part 2: search and retrieve one user

```
// Example: GET /users/search?name=Ja
app.get('/users/search', async (req, res) => {
  const name = req.query.name;
  if (typeof name !== 'string' || name.trim() === '') {
    return res.status(400).json({ error: 'Provide a name query' });
  }

  try {
    const pattern = `%${name.trim()}%`;
    const rows = await sql`
      SELECT user_id, name, age
      FROM users
      WHERE LOWER(name) LIKE LOWER(${pattern})
      ORDER BY user_id
    `;
    return res.json(rows);
  } catch (error) {
    console.error('Search failed:', error);
    return res.status(500).json({ error: 'Could not search users' });
  }
});

// Example: GET /users/1
app.get('/users/:id', async (req, res) => {
  const userId = Number(req.params.id);
  if (!Number.isSafeInteger(userId) || userId < 1) {
    return res.status(400).json({ error: 'ID must be a positive integer' });
  }

  try {
    const rows = await sql`
      SELECT user_id, name, age
      FROM users
```

```
      WHERE user_id = ${userId}
    `;

    if (rows.length === 0) {
      return res.status(404).json({ error: 'User not found' });
    }
    return res.json(rows[0]);
  } catch (error) {
    console.error('Get one user failed:', error);
    return res.status(500).json({ error: 'Could not retrieve user' });
  }
});
```

**Why this works.** The `:id` part of `/users/:id` is a *route parameter*. For `/users/1`, Express makes `req.params.id` equal to `'1'` (a string). `Number(...)` converts it to a number, and the condition rejects missing/invalid ID values.

**Step 3.** Add the POST route below the previous routes. Note that the server *does not* pass the entire request body directly to `sql(user)`: it builds a new object containing only the approved columns.

Listing 7: server.js – part 3: insert a new user and start the server

```
// Example: POST /users with JSON { "name": "John", "age": 30 }
app.post('/users', async (req, res) => {
  const { name, age } = req.body ?? {};

  if (
    typeof name !== 'string' ||
    name.trim().length === 0 ||
    name.trim().length > 100 ||
    !Number.isInteger(age) ||
    age < 0 || age > 130
  ) {
    return res.status(400).json({
      error: 'Provide a non-empty name (max 100 characters) '
        + 'and an integer age between 0 and 130'
    });
  }

  const user = { name: name.trim(), age };

  try {
    const rows = await sql`
      INSERT INTO users ${sql(user)}
      RETURNING user_id, name, age
    `;
    return res.status(201).json(rows[0]);
  } catch (error) {
    console.error('Create user failed:', error);
    return res.status(500).json({ error: 'Could not create user' });
  }
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

**Step 4.** Save `server.js` (**Ctrl+S** / **Cmd+S**). Check that the file contains only *one* `app.listen(...)` statement, at its end.

**Learning checkpoint**

**Checkpoint D** You now have a complete server with six routes. In `server.js`, locate one example of each: `await`, `sql`...``, `${userId}`, `sql(user)`, `res.json(...)`.

# 6 Part E – Run and test each route

## 6.1 Start the server

**Step 1.** Check that the PostgreSQL service is still running and the database `workshop8_db` exists.

**Step 2.** In the project's terminal, type:

```
npm start
```

**Expected result.** You should see:

```
> node server.js
Server running at http://localhost:3000
```

Leave this terminal open. The process runs until you press **Ctrl+C**.

**Helpful note**

If you edit server code later, stop the running process with **Ctrl+C**, save the file and run `npm start` again. If your chosen port is already occupied, change `PORT` in `.env` and restart.

## 6.2 Test 1: Verify the server is reachable

**Step 3.** Open a browser and visit `http://localhost:3000/`.

**Expected result.** You see JSON similar to:

```
{ "message": "Welcome to Workshop 8!" }
```

**What you learned:** Express can answer an HTTP GET request without needing to query the database.

## 6.3 Test 2: Verify the PostgreSQL connection

**Step 4.** Open `http://localhost:3000/health`.

**Expected result.** The response is similar to:

```
{
  "message": "Database connected",
  "result": { "ok": 1 }
}
```

If you see HTTP 500, inspect the VS Code terminal for a connection error and consult the troubleshooting section.

## 6.4 Test 3: Read all records

**Step 5.** Open http://localhost:3000/users.

**Expected result.** On a fresh database, you should see:

```
[
  { "user_id": 1, "name": "Jane", "age": 27 },
  { "user_id": 2, "name": "Joe", "age": 26 }
]
```

**Explain it:** The route runs `SELECT`, receives database rows, and sends the rows as JSON using `res.json(rows)`.

## 6.5 Test 4: Read one record with a bound parameter

**Step 6.** Open http://localhost:3000/users/1.

**Expected result.** You get the user with `user_id = 1`, for example:

```
{ "user_id": 1, "name": "Jane", "age": 27 }
```

**Step 7.** Try these addresses and compare the outcome:

- http://localhost:3000/users/99999 → HTTP 404 and `"User not found"` (unless that ID exists).
- http://localhost:3000/users/abc → HTTP 400 because `abc` is not a valid numeric ID.

**Explain it:** Successful requests produce a matching object; invalid input produces **400 Bad Request**; a valid but missing ID produces **404 Not Found**.

## 6.6 Test 5: Search by a name

**Step 8.** Open http://localhost:3000/users/search?name=Ja.

**Expected result.** The result includes Jane if she is in your table. Try `name=Jo` and `name=xyz` to see how the output changes.

**Helpful note**

The example uses `LIKE`: `%` matches any sequence of characters. This gives a beginner-friendly substring search. In a production API, consider query length limits and special handling of the `%` and `_` wildcard characters if you need a strictly literal search.

## 6.7 Test 6: Insert a user through POST

A web browser's address bar normally sends GET requests, not POST. Use VS Code's **REST Client** extension or Postman for this step.

**Step 9.** In VS Code, open **Extensions**, find **REST Client**, and install it (if it is not installed already).

**Step 10.** Create a file named `requests.http` and type:

Listing 8: requests.http – reusable requests for every route

```
### Welcome route
```

```
GET http://localhost:3000/

### Database health check
GET http://localhost:3000/health

### All users
GET http://localhost:3000/users

### Search for names containing Ja
GET http://localhost:3000/users/search?name=Ja

### One user
GET http://localhost:3000/users/1

### Create a user
POST http://localhost:3000/users
Content-Type: application/json

{
  "name": "John",
  "age": 30
}

### Try invalid input
POST http://localhost:3000/users
Content-Type: application/json

{
  "name": "",
  "age": -5
}
```

**Step 11.** Click the **Send Request** link above the `POST /users` section with John's data. Inspect the response pane.

**Expected result.** The response status is **201 Created**, and the response body contains something like:

```
{ "user_id": 3, "name": "John", "age": 30 }
```

`user_id` may differ if the database already contains other users.

**Step 12.** Send `GET /users` again. Verify that John now appears in the returned list. Then send the invalid POST request. Its expected status is **400 Bad Request** and no record should be added.

**Helpful note**

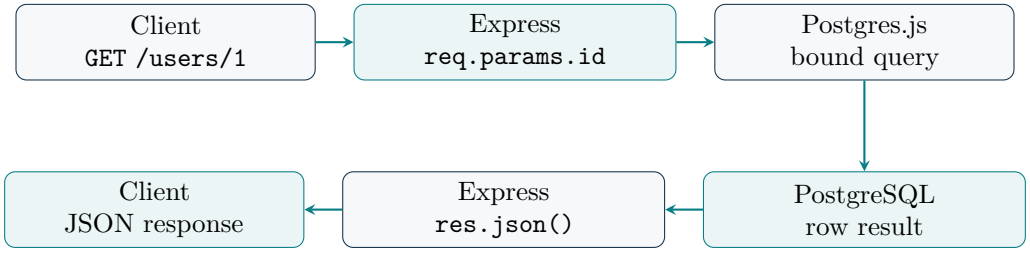
**Postman alternative:** Create a request with method **POST**, URL `http://localhost:3000/users`, choose **Body** → **raw** → **JSON**, paste `{"name":"John","age":30}` and click **Send**.

## 7 Part F – Trace what happens behind the scenes

Consider the request `GET /users/1`. Follow its journey:

1. A client sends an HTTP GET request to `http://localhost:3000/users/1`.
2. Express matches the `/users/:id` route and captures the path parameter `'1'`.
3. JavaScript converts the parameter to `1` and checks it is a positive integer.

4. Postgres.js executes the tagged SQL template with the ID as a bound parameter.
5. PostgreSQL returns a matching row (if one exists).
6. Express sends `res.json(rows[0])`; the client displays the JSON.




### 7.1 Mini experiment: inspect a SQL value

For a short experiment, insert this statement immediately after the ID conversion in the GET-by-ID route:

```
console.log('Requested user ID:', userId);
```

Save the file, restart with `npm start`, and open `/users/2`. Check the terminal. You should see `Requested user ID: 2`. Remove the temporary statement after you understand its purpose.

## 8 Part G – Troubleshooting for beginners

When your code does not run, read the *first useful error message* and compare it with this table.

| Message or symptom | What to check / how to fix it |
|---|---|
| `node` or `npm` not recognised | Install Node.js LTS, close and reopen the terminal, run `node -version` again. |
| `Cannot use import statement outside a module` | Confirm that `package.json` has `"type": "module"`; run `npm pkg set type=module`. |
| `ERR_MODULE_NOT_FOUND` | Install dependencies with `npm install express postgres dotenv`; check your current folder and the exact filename `db.js`. |
| `ECONNREFUSED` | PostgreSQL service may not be running, or `PGHOST`/`PGPORT` is wrong. Open pgAdmin and check the port. |
| `password authentication failed` | Check `PGUSER` and `PGPASSWORD` in `.env`. Save and restart the server. |
| `database ... does not exist` | Verify `PGDATABASE=workshop8_db` and create that database in pgAdmin. |
| `relation "users" does not exist` | Confirm you ran `schema.sql` in the *workshop8_db* database, not in *postgres*. |
| `EADDRINUSE` | Port 3000 is in use. Stop another running server or change `PORT` in `.env`, restart, and update test URLs. |
| `Cannot GET /users` | Check the route spelling, save `server.js`, restart, and ensure your browser URL and port match. |
| HTTP 400 when POSTing | Confirm the request body is valid JSON, `Content-Type: application/json` is present, `name` is non-empty, and `age` is an integer between 0 and 130. |
| HTTP 404 on a valid ID | The record may genuinely not exist. Run `SELECT * FROM users ORDER BY user_id;` in pgAdmin to inspect IDs. |

| Message or symptom | What to check / how to fix it |
|---|---|
| Old response after editing code | Save the file and restart `npm start`. |

# 9 Part H – Independent practice

Complete these exercises without copying an answer from another student. Test each change before moving to the next exercise.

## 9.1 Exercise 1: Query a new user (easy)

**Task:** Send `POST /users` with name `Aisha` and age `21`. Then retrieve her new ID through `GET /users`, and open `GET /users/<her-id>`.

**Evidence to record:** The POST status code, the returned ID, and the GET-by-ID response.

## 9.2 Exercise 2: Filter users by minimum age (medium)

**Task:** Add `GET /users/adults` before the `/users/:id` route. It should return all users aged 18 or older, sorted by ID.

**Hints:**

1. Copy the structure of `GET /users` to make a new route.
2. Use SQL `WHERE age >= 18` before `ORDER BY user_id`.
3. Test `GET http://localhost:3000/users/adults` in `requests.http`.

**Success condition:** John (30) appears, while a newly added user aged 16 does not.

## 9.3 Exercise 3: Use a bound query value (medium)

**Task:** Create `GET /users/older-than/:age`. Return users whose `age` is greater than the supplied age. Validate that `:age` is an integer in the range 0–130.

**Hints:** Use `Number(req.params.age)` and this query pattern:

```
const rows = await sql`
  SELECT user_id, name, age
  FROM users
  WHERE age > ${minAge}
  ORDER BY age
`;
```

**Success condition:** `GET /users/older-than/26` returns people older than 26 and rejects `/users/older-than/abc` with HTTP 400. Place your route before `/users/:id` for clarity.

## 9.4 Exercise 4: Reflect on security (short answer)

Explain in two or three sentences why `sql‘WHERE user_id = ${userId}‘` is safer than constructing a SQL statement by concatenating untrusted request text. Distinguish *SQL injection* from *XSS*.

# 10 Part I – Self-assessment and submission

**Student name:** _____ **Student ID:** _____

- ☐ I created `workshop8_db` and the `users` table.
- ☐ I installed `express`, `postgres` and `dotenv`.
- ☐ `npm start` displays the server URL without errors.
- ☐ `GET /health` confirms that PostgreSQL can be queried.
- ☐ `GET /users` returns a JSON array.
- ☐ `GET /users/1` demonstrates a bound query parameter.
- ☐ `POST /users` creates a record and returns HTTP 201.
- ☐ Invalid POST input returns HTTP 400 without adding a record.
- ☐ I completed at least two independent-practice exercises.

**Suggested submission:** Provide `schema.sql`, `db.js`, `server.js`, `requests.http`, a screenshot of one successful GET request, a screenshot of a successful POST request and a brief written response to Exercise 4. **Do not submit** `.env`, real passwords or the `node\_modules` folder.

## 10.1 Reflection prompts

1. What is the difference between an Express route and a PostgreSQL query?
2. What does `await` do in `const rows = await sql‘...‘;`?
3. What does `RETURNING` add to an `INSERT` query?
4. Why should we select allowed object fields before calling `sql(user)`?
5. What is the difference between status codes 201, 400, 404 and 500?

# 11 Quick-reference sheet

| Syntax | Meaning |
|---|---|
| `npm start` | Runs the `start` script in `package.json`. |
| `import sql from ’./db.js’` | Loads the shared database client. |
| `await sql‘SELECT ...‘` | Executes a SQL query and waits for its result. |
| `${userId}` inside `sql‘...‘` | Binds a JavaScript value as SQL data. |
| `sql(user)` | Builds a safe insert value fragment from named object fields. |
| `RETURNING user_id` | Returns generated/selected values after inserting. |
| `req.params.id` | Gets a named URL path parameter. |
| `req.query.name` | Gets a URL query-string value. |
| `req.body` | Gets request body data (after the JSON middleware). |
| `res.status(201).json(...)` | Sends JSON with the HTTP status 201 Created. |